



University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Report Number: 01

Experiment Title: First Come First Serve (FCFS)

Submitted by:

Name : Nabihah Afrin

Student ID : 23101125

Section : C1

Submitted to:

Atia Rahman Orhi

Lecturer, Department of Computer Science and Engineering

Date of Submission: 13 Aug, 2026

Problem Statement

Implement FCFS (First Come First Serve) CPU Scheduling for the following scenario where the number of processes = 5, and the process information is given below:

pid	AT	BT
P4	1	2
P2	2	4
P1	3	5
P3	4	3
P5	5	3

Calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT. Also represent the process execution sequence.

Code:

```
pid = ["P4", "P2", "P1", "P3", "P5"]
AT = [1, 2, 3, 4, 5]
BT = [2, 4, 5, 3, 3]

CT = []
TAT = []
WT = []

ct = 0

for i in range(5):
    if ct < AT[i]:
        ct = AT[i]

    ct = ct + BT[i]
    CT.append(ct)

for i in range(5):
    tat = CT[i] - AT[i]
    TAT.append(tat)

    wt = tat - BT[i]
```

```

WT.append(wt)

print("PID\tAT\tBT\tCT\tTAT\tWT")

for i in range(5):
    print(f"{pid[i]}\t{AT[i]}\t{BT[i]}\t{CT[i]}\t{TAT[i]}\t{WT[i]}")

avg_tat = sum(TAT) / 5
avg_wt = sum(WT) / 5

print("\nAverage TAT =", avg_tat)
print("Average WT =", avg_wt)

print("\nExecution Sequence:")
print("P4 -> P2 -> P1 -> P3 -> P5")

```

Screenshot:

The screenshot shows an IDE with a Python file named `FCFS.py`. The code calculates the average Turnaround Time (TAT) and Waiting Time (WT) for five processes (P1 to P5) using the First-Come-First-Served (FCFS) scheduling algorithm. The output displays a table of process data, the calculated averages, and the execution sequence.

```

C:\Users\User\Desktop\proj\orthi miss\.venv\Scripts\python.exe "C:\Users\User\Desktop\proj\orthi miss\FCFS.py"
PID AT BT CT TAT WT
P4 1 2 3 2 0
P2 2 4 7 5 1
P1 3 5 12 9 4
P3 4 3 15 11 8
P5 5 3 18 13 10

Average TAT = 8.0
Average WT = 4.6

Execution Sequence:
P4 -> P2 -> P1 -> P3 -> P5

Process finished with exit code 0

```

Discussion

While implementing FCFS, I initially had a problem with some invisible formatting characters that affected the output arrangement. After fixing them, the program gave the correct results.

FCFS is simple and easy to implement because processes are executed in arrival order. It is non-preemptive, meaning a running process cannot be stopped until it finishes. FCFS usually avoids starvation because every process gets its turn, but a long process can make other processes wait for a long time. Overall, this lab helped me understand the basic working of FCFS scheduling.

Conclusion

In this lab, the FCFS CPU scheduling algorithm was successfully implemented using Python. The program calculated the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and their average values, along with the process execution sequence. The implementation demonstrated that FCFS is simple, easy to understand, and non-preemptive. Overall, the lab provided a practical understanding of how FCFS scheduling works and how scheduling performance can be measured.